

TENSOR DECOMPOSER

An Interactive High-Dimensional Multiway Data Analysis and Benchmarking
System

FINAL TECHNICAL REPORT

SE 801 — Software Project Lab III

Submitted By:

Md. Kamrul Hasan

Roll: BSSE-1418

Institute of Information Technology (IIT), University of Dhaka

Supervised By:

Mohd. Zulfiquar Hafiz,

Professor,

Institute of Information Technology (IIT), University of Dhaka

Submitted To:

SPL 3 Program Committee,

Institute of Information Technology (IIT), University of Dhaka

Date of Submission: 10 October 2026

Letter of Transmittal

Date: 10 October 2026

To,

The SPL 3 Program Committee & Exam Committee

Institute of Information Technology (IIT), University of Dhaka

Subject: Final Technical Report Submission on 'Tensor Decomposer'

Dear Sir/Madam,

I am pleased to submit the Final Technical Report for my Software Project Lab III (SE 801) project, titled "Tensor Decomposer: An Interactive High-Dimensional Multiway Data Analysis and Benchmarking System."

This report comprehensively documents the completed system: its requirements, architecture, algorithms, PuzzleTensor preprocessing integration, the standalone tensordecomp Python library, testing outcomes and cloud deployment on Vercel. The system has been fully implemented and is publicly accessible.

I sincerely thank my supervisor Mohd. Zulfiqar Hafiz sir for continued guidance and the SPL 3 committee for their support. I look forward to your evaluation and valuable feedback.

Sincerely,

Md. Kamrul Hasan

BSSE1418

IIT, University of Dhaka

Supervisor Signature_____

Acknowledgement

First and foremost, I express my deepest gratitude to Almighty Allah for providing me the strength, endurance, and capability to complete this project successfully.

I extend my sincere appreciation to my project supervisor, Mohd. Zulfiqar Hafiz, Professor, Institute of Information Technology (IIT), University of Dhaka, for his invaluable guidance, scholarly advice, and constructive reviews throughout this project.

I am also deeply grateful to the faculty members and SPL 3 Program Committee of IIT for providing a rich academic environment. Special thanks to the researchers whose papers—particularly Park et al. (KDD 2025) on PuzzleTensor, Kolda & Bader (SIAM 2009) and V. Oseldets (SIAM 2011) on tensor decompositions—formed the theoretical backbone of this work.

I also thank the team at Vercel for providing free cloud deployment infrastructure that made the application publicly accessible.

Abstract

Modern datasets across scientific disciplines—including deep learning, image processing, signal processing, neuroimaging (EEG/fMRI), chemometrics, and climate science—are inherently multi-dimensional and naturally structured as higher-order tensors. Traditional matrix factorization methods such as PCA or SVD require flattening (matricizing) multiway data, which destroys intrinsic higher-order correlations and multilinear structure. Conversely, operating directly on raw high-dimensional tensors introduces substantial computational complexity, memory overhead, and numerical stability challenges.

This report presents Tensor Decomposer, a fully implemented, publicly deployed web-based analytical platform that decomposes, benchmarks, and visualizes higher-dimensional tensor datasets. The platform implements four core tensor decomposition algorithms—CP Decomposition via Alternating Least Squares (CP-ALS), Tucker Decomposition via Higher-Order Orthogonal Iteration (HOOI), Higher-Order Singular Value Decomposition (HOSVD) and Tensor Train (TT) Decomposition. Each tensor algorithm is augmented with the PuzzleTensor preprocessing engine, inspired by Park et al. (KDD 2025), which applies greedy cyclic hyperslice shifting to reduce the effective multilinear rank of input tensors before decomposition.

Additionally, the project delivers the `tensordecomp` Python library: a standalone, pip-installable package with zero heavy dependencies (NumPy only) exposing all decomposition algorithms, analysis utilities, and the PuzzleTensor engine via a clean public API.

Keywords: *tensor, tensor decomposition, CP-ALS, Tucker, HOOI, HOSVD, Tensor Train, PuzzleTensor, FLOPs benchmarking, web application, Python library, multiway data analysis, numerical computing, Vercel deployment*

Table of Contents

1. Introduction	6
1.1 Project Title	6
1.2 Problem Statement	6
1.4 Scope & Deliverables	6
2. Requirements Analysis	7
2.1 Functional Requirements	7
2.2 Non-Functional Requirements	8
2.3 Stakeholders	8
3. State Transition Diagram	9
4. Sequence Diagram	10
4. Architectural Design	11
4.1 Architectural Context Diagram	11
5. Component Design	13
6. Deployment Diagram	14
7. User Manual	15
7.1 System Requirements	15
7.2 Web Application Layout	16
7.2.1 Tensor Input	17
7.2.2 Tensor Decompose	19
7.2.3 Tensor Visualize	19
7.2.4 Decompose Benchmark	23
7.2.5 Decompose Restruct	24
7.2.6 Compare All Methods	25
7.3 Library Installation	26
7.3.1 Library Functions	27
7.3.1 Library Implementations	28
8. Public Accessible	30
8.1 Version Control Repository	30
8.2 Live url	30
8.3 Library	30
9. Test Plan	30
9.1 Testing Strategy & Objectives	30
9.2 Test Cases	31
9.2.1 Automated Unit, Integration & API Test Cases	31

9.2.2 User Acceptance Test Cases	35
9.2.3 Library Test Cases	38
10. Conclusion	42
11. References	43

1. Introduction

1.1 Project Title

Tensor Decomposer: An Interactive High-Dimensional Multiway Data Analysis and Benchmarking System

1.2 Problem Statement

In modern data-driven fields—computer vision, deep learning, signal processing, neuroimaging, and climate science—data naturally manifests as multi-dimensional tensors rather than traditional 2D matrices. Color video sequences have dimensions (Height $\times$ Width $\times$ Frames $\times$ Channels); fMRI scans are 4D; rainfall data forms a 3D tensor of (Years $\times$ Weeks $\times$ Stations).

Conventional matrix reduction techniques (PCA, SVD) require flattening tensors into 2D matrices, destroying intrinsic multiway correlations and spatial patterns. Conversely, computing higher-order tensor factorizations introduces substantial complexity, memory overhead, and numerical stability risks. Furthermore, no unified, publicly accessible, browser-based tool exists for researchers and students to interactively explore, benchmark, and visualize tensor decompositions without local installation.

1.4 Scope & Deliverables

- Web Application hosted at <https://td4.vercel.app>.
- Four tensor decomposition algorithms (CP, Tucker, HOSVD, Tensor Train).
- Four PuzzleTensor-augmented variants (cp_puzzle, tucker_puzzle, hosvd_puzzle, tensor_train_puzzle).
- tensordecomp Python library with public API, editable-installable via pip.
- JSON result exporting and downloading from the web interface.

2. Requirements Analysis

2.1 Functional Requirements

ID	Feature	Description
FR-01	Tensor Input	Accept N-dimensional tensor input via manual JSON text entry or file upload (.txt, .json).
FR-02	Input Validation	Parse, validate shape consistency, data types and detect malformed arrays; display descriptive error messages.
FR-03	Algorithm Selection	Allow selection of 8 methods: CP, Tucker, HOSVD, TT, CP+Puzzle, Tucker+Puzzle, HOSVD+Puzzle, TT+Puzzle.
FR-04	Rank Configuration	Accept rank input per algorithm: scalar R for CP; multilinear ranks $[R_1, R_2, R_3]$ for Tucker/HOSVD; TT-ranks for Tensor Train.
FR-05	Decompose Action	Execute selected algorithm and display factor matrices, weights, core tensors, or TT cores.
FR-06	Benchmark Action	Measure single-run execution time (ms), estimate FLOPs, compute complexity formula, compression ratio.
FR-07	Compare Action	Run all supported algorithms on the same tensor and display a comparative table.
FR-08	Error Metrics	Compute MAE, RMSE, Absolute Error, Relative Error between original and reconstructed tensor.
FR-09	SVG Visualization	Render 3D isometric decomposition equation diagrams and 2D matrix heatmaps.

FR-10	JSON Export	Allow downloading decomposition results as a JSON file.
FR-11	PuzzleTensor	Apply greedy hyperslice shifting pre-processing before decomposition; invert shifts on reconstruction.
FR-12	Dark/Light Theme	Toggle between dark and light UI themes; persist preference in localStorage.

2.2 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	API response ≤ 5 seconds for tensors up to $50 \times 50 \times 50$; FLOPs estimation < 10 ms.
NFR-02	Accuracy	Algorithm output relative error within 1×10^{-6} of mathematical ground truth for synthetic tensors.
NFR-03	Reliability	Zero application crashes on invalid input; all errors handled with descriptive messages.
NFR-04	Accessibility	Publicly deployed; no registration required.
NFR-05	Portability	tensordecomp library compatible with Python ≥ 3.10 ; requires only NumPy ≥ 1.26 .
NFR-06	Maintainability	Modular architecture with separate algorithm modules; 34+ unit tests for regression prevention.
NFR-07	Security	Input sanitization for all POST data; file upload restricted to UTF-8 text files.

2.3 Stakeholders

Stakeholder	Role / Interest
Students & Researchers	Primary users seeking to decompose and analyze tensor data for coursework or research.

Data Scientists	Practitioners who need benchmarking comparisons to select algorithms for real-world tasks.
ML/DL Engineers	Engineers exploring tensor compression for model parameter reduction.
Open-Source Community	Users of the tensordecomp library for local scripting and experimentation.

3. State Transition Diagram

A State Transition Diagram is a graphical representation of a finite state machine, used to model objects with a finite number of states and their interactions with the outside world through state changes triggered by events. It consists of nodes representing states and directed arcs representing transitions labeled with event names.

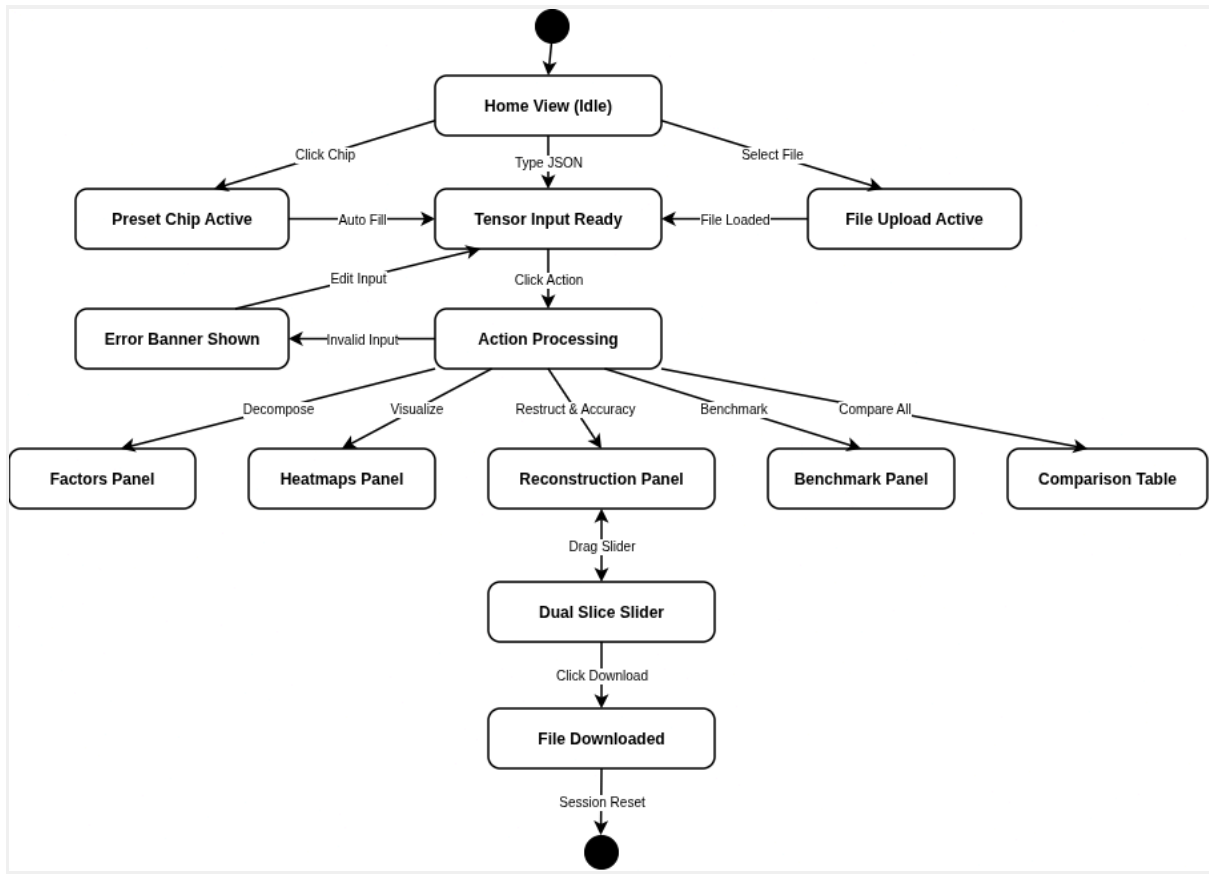


Figure - 1: Sequence Diagram

4. Sequence Diagram

Sequence diagram shows process interactions arranged in time sequence. This diagram depicts the processes and objects involved and the sequence of messages exchanged as needed to carry out the functionality.

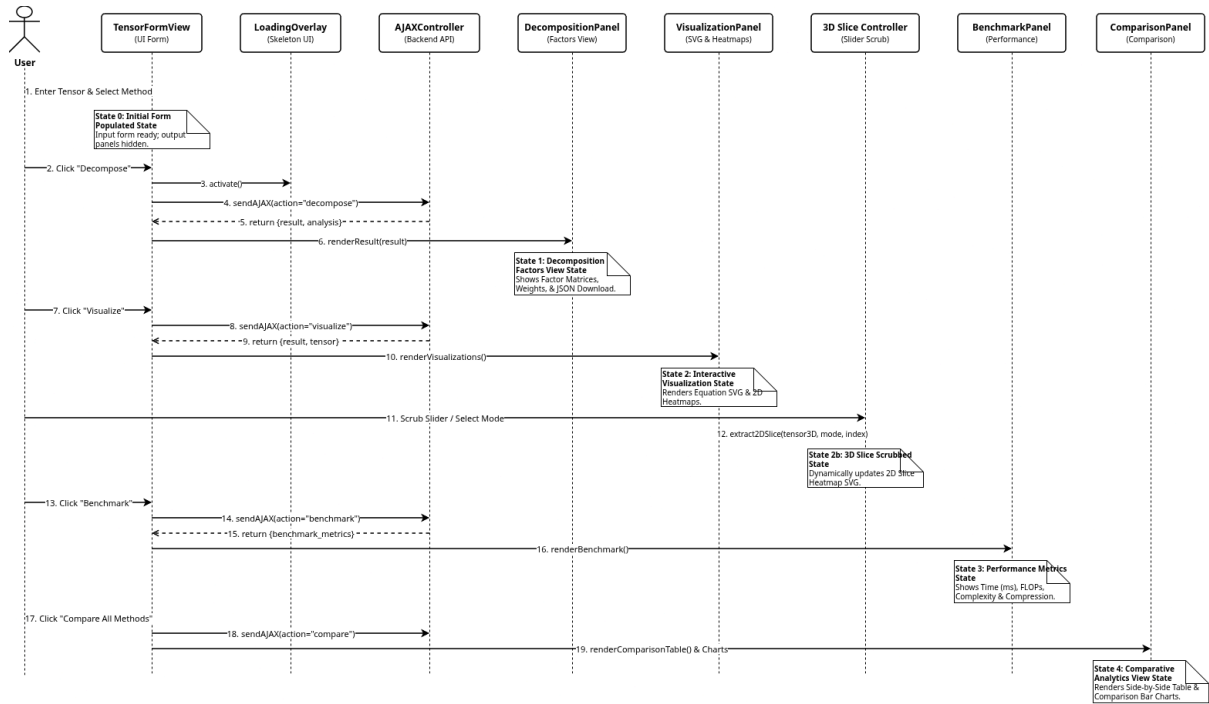


Figure - 2: Sequence Diagram

4. Architectural Design

Architectural design gives an overview of how a system works and its interactions with modules. It connects modules and improves our knowledge about the system.

4.1 Architectural Context Diagram

The Context Architectural Diagram provides a high-level overview of the proposed tensor decomposition system and its interaction with external users. It represents the system as a single unified component without showing its internal modules or implementation details. The diagram illustrates how users provide tensor data, configure decomposition methods and interact with the system to obtain results and downloadable artifacts. This diagram helps establish the system boundary and clearly identifies the primary inputs and outputs of the application.

In the system architecture, the entities interacting with the Tensor Decomposer System are classified into superordinate, subordinate, and peer/interfacing systems:

1. **Superordinate Systems (Enclosing Environment):** Superordinate systems represent the higher-level operational and runtime environments that govern, host and provide execution boundaries for the system.
2. **Subordinate Systems (Dependent Components & Engines):** Subordinate systems are internal or controlled modules that execute specialized, low-level tasks delegated by the central system.
3. **Peer & Interfacing Systems (External Actors & Consumers):** Peer and external systems interact across the architectural boundary without hierarchical containment.

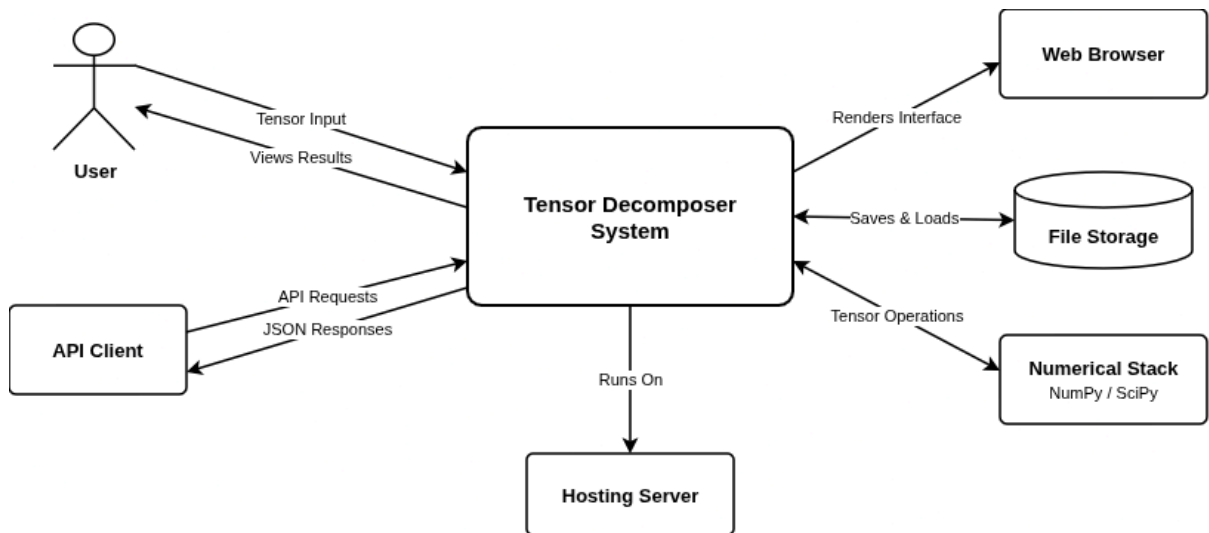


Figure 3: Architectural Context Diagram

5. Component Design

The component-level design of the Tensor Decomposer System follows a modular, layered architecture emphasizing high cohesion, loose coupling, and strict separation of concerns. The system is decomposed into four primary functional modules, each enclosing specialized UML components.

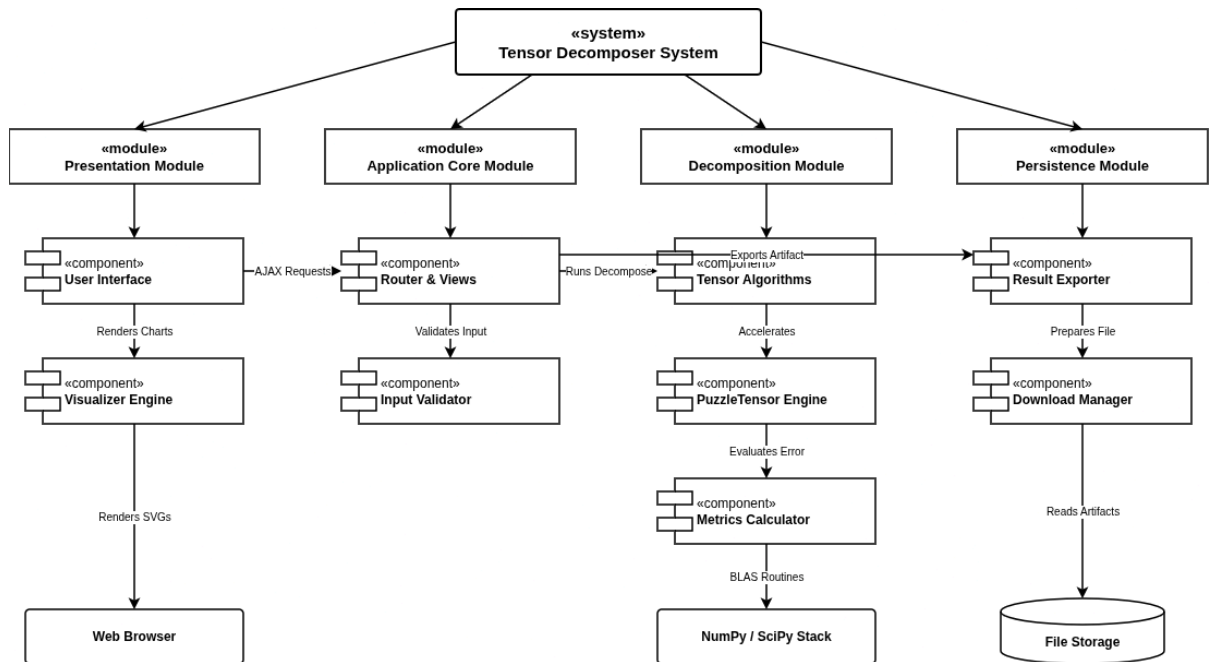


Figure - 4: Top Level Component

6. Deployment Diagram

The deployment diagram illustrates the physical mapping of the Tensor Decomposer across a client workstation, an application server and dedicated file storage. The client-side web browser executes the interactive frontend interface, communicating with the server over secure HTTPS to exchange tensor payloads and visualization requests. On the server tier, a WSGI gateway feeds incoming requests to the Django application and the underlying tensor decomposition engine running on an optimized numerical computing stack. Decomposed factor artifacts, reconstructed models and temporary file uploads are persisted directly to the attached storage volume for reliable user retrieval.

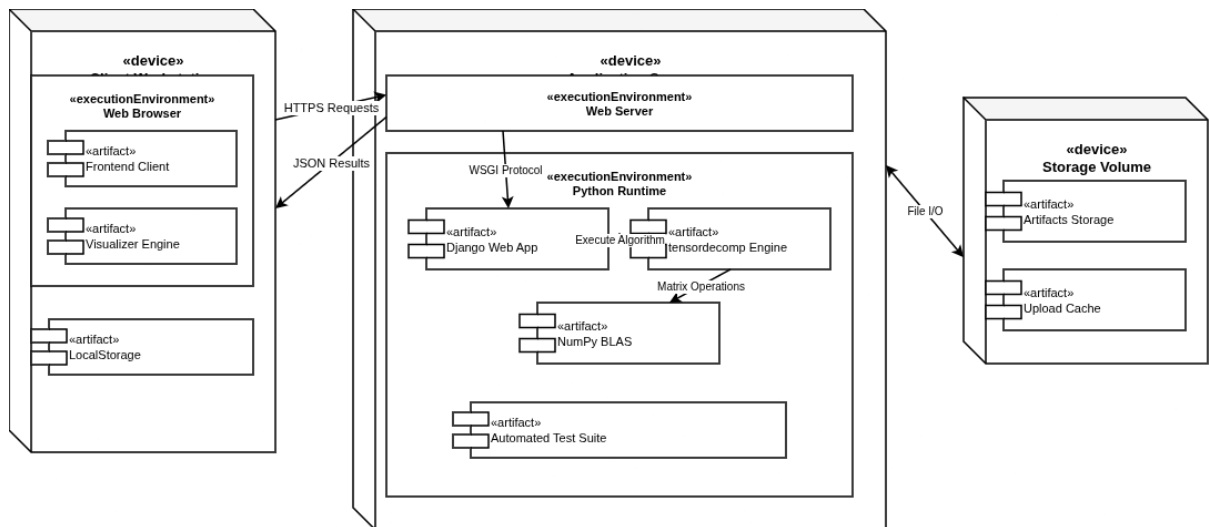


Figure - 5: Deployment Diagram

7. User Manual

The manual shows systematic steps of using web application and tensordecomp library. The web application allows users to upload or insert tensors in the app and returns decomposed tensors, accuracy, compression and benchmarking information. The app also shows visualization for decomposed output. The library achieves similar functionalities in the local environment.

7.1 System Requirements

Web browser	Chromium or Firefox based browser (Brave, Chrome, Firefox, Tor, Mullvad, Edge etc)
Internet	Active internet connection to connect with web app
File Format	.txt, .json
Operating System	Linux, Windows, MacOS
Access URL	https://td4.vercel.app/
Library Install Command	<code>pip install tensordecomp</code>

7.2 Web Application Layout

The web application has several key panels

1. **Header:** This area shows the web app name, its objective and light/dark toggle mode button.
2. **Left panel:** It takes manual or copied tensor as input. It can also take random fixed size input for auto inserts.
3. **Right panel:** Here we define in which method we would like to decompose our tensor. There are eight decomposition methods. Users can insert an input file from here. Users can define the rank of decomposed outputs. Functionalities of the tensor decomposer are also given here. Users can choose to decompose, visualize, reconstruct, benchmark or compare every single method from this panel.
4. **Bottom panel:** Output of all the methods and functionalities will be presented at the bottom. Decomposed tensors will have a downloadable json button. Visualization will show modes of outputs in a very user friendly way. Benchmark shows complexity, time and compression of methods. Reconstruction shows relative error, absolute error, root mean squared error of the methods.

Tensor Decomposer

An Interactive High-Dimensional Multiway Data Analysis and Benchmarking System

Configuration & Inputs

Define input tensor parameters and algorithm options

TENSOR INPUT (JSON ARRAY)

Auto Insert: 3×3×3 (3D) 4×3×2 (3D) 4D (2+) 4D (3×2×3×2)

```
[[[1, 2], [3, 4]], [[5, 6], [7, 8]]]
```

METHOD / ALGORITHM

CP Decomposition

TARGET RANKS (OPTIONAL)

Single scalar R (Min: 1, Max: 50)

CP requires a single rank R across all factor matrices. Fixed bounds : Min = 1, Max = 50.

UPLOAD TENSOR FILE

Choose File | No file chosen

Decompose

Visualize

Benchmark & Compression

Restruct & Accuracy

Compare All Methods

Figure - 6: Full Page

7.2.1 Tensor Input

Users can manually insert a tensor in the left panel ‘Tensor Input (JSON Array)’. Users can auto insert some random tensors if they want. There is an option to enter tensors via file in the right panel ‘Upload Tensor File’. In the case of uploading files, a .json or .txt will be required. In any case, the right format of tensor with number elements and proper dimensions will be required.

Configuration & Inputs

Define input tensor parameters and algorithm options

TENSOR INPUT (JSON ARRAY)

Auto Insert: 3×3×3 (3D) 4×3×2 (3D) 4D (2+) 4D (3×2×3×2)

```
[
  [
    [6, 3],
    [7, 8],
    [3, 8]
  ],
  [
    [7, 4],
    [4, 4],
    [6, 1]
  ],
  [
    [4, 7],
    [9, 9],
    [1, 6]
  ],
  [
    [7, 2],
    [5, 3],
    [5, 6]
  ]
]
```

METHOD / ALGORITHM

CP Decomposition

TARGET RANKS (OPTIONAL) Rank R (Single Integer)

Single scalar R (Min: 1, Max: 24)

CP requires a single rank R across all factor matrices. Fixed bounds for 4×3×2 tensor: Min = 1, Max = 24.

UPLOAD TENSOR FILE

Choose File No file chosen

Decompose Visualize

Benchmark & Compression Restruct & Accuracy

Compare All Methods

Figure - 7: Tensor Manual Input

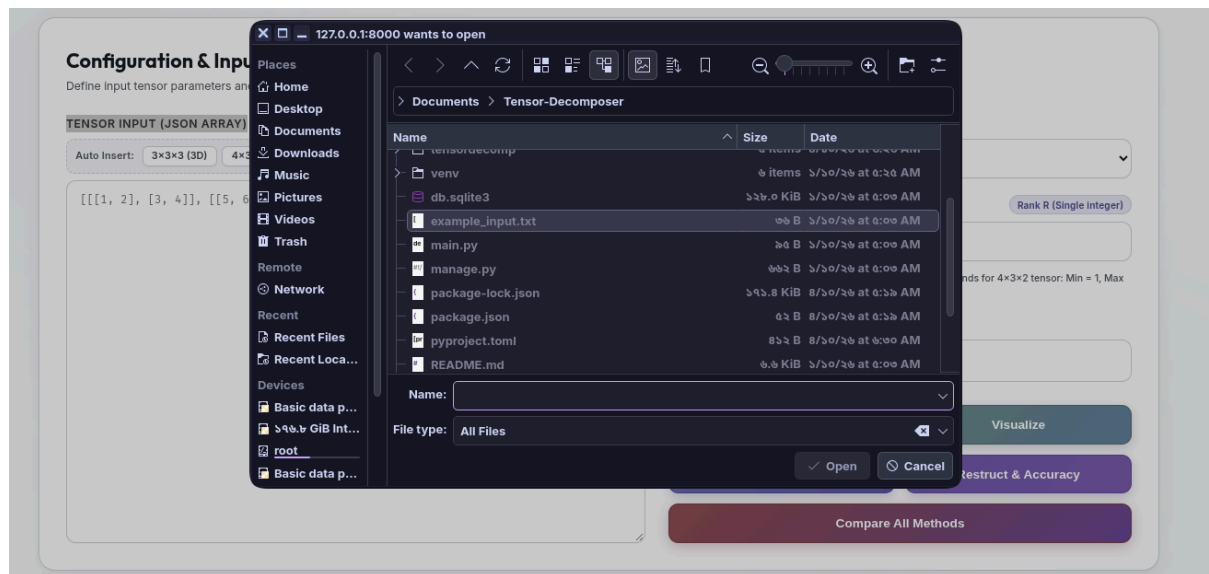


Figure - 8: Tensor File Input

7.2.2 Tensor Decompose

Steps:

1. Enter a tensor via manual insertion, random input in the left panel or file input in the right panel.
2. Press the Decompose button in the right panel.
3. Resulting tensors will be shown in the bottom section.

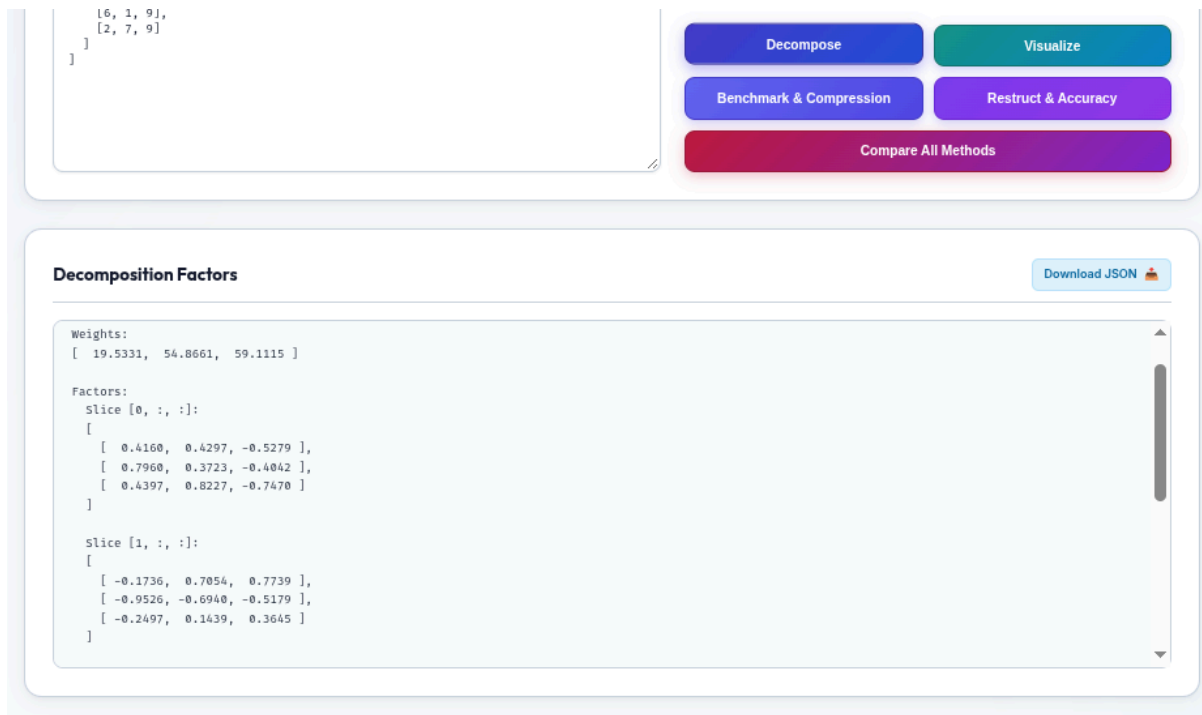


Figure - 9: Decompose

7.2.3 Tensor Visualize

Steps:

1. Enter a tensor via manual insertion, random input in the left panel or file input in the right panel.
2. Press the Visualize button in the right panel.
3. Output visualization will be shown in the bottom section.

Output Visualization

Visualizing: CP

Decomposition Equation Structure

Scaled ratio-wise based on input tensor and decomposed factors



Original Input Tensor ($4 \times 3 \times 2$)

Mode 0 Slice [1/4] — Shape: 3×2

Mode 0 Slice [1/4] — Shape: 3×2



Component Weights Spectrum (λ)



Factor Matrix - Mode 1 (4×2)

Shape: 4×2



Factor Matrix - Mode 2 (3×2)

Shape: 3×2



Factor Matrix - Mode 3 (2×2)

Shape: 2×2



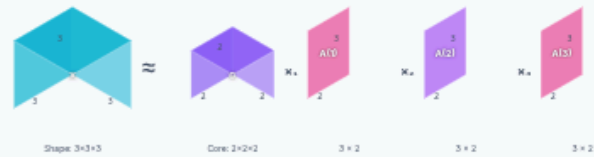
Figure -10: Visualize (CP)

Output Visualization

Visualizing: TUCKER

Decomposition Equation Structure

Scaled ratio-wise based on input tensor and decomposed factors



Original Input Tensor (3x3x3)

Mode 0 Slice [1/3] — Shape: 3x3



Factor Matrix - Mode 1 (3x2)



Factor Matrix - Mode 2 (3x2)



Factor Matrix - Mode 3 (3x2)



Core Tensor G (2x2x2)

Mode 0 Slice [1/2] — Shape: 2x2



Figure -11: Visualize (Tucker)



Figure -12: Visualize (HOSVD)

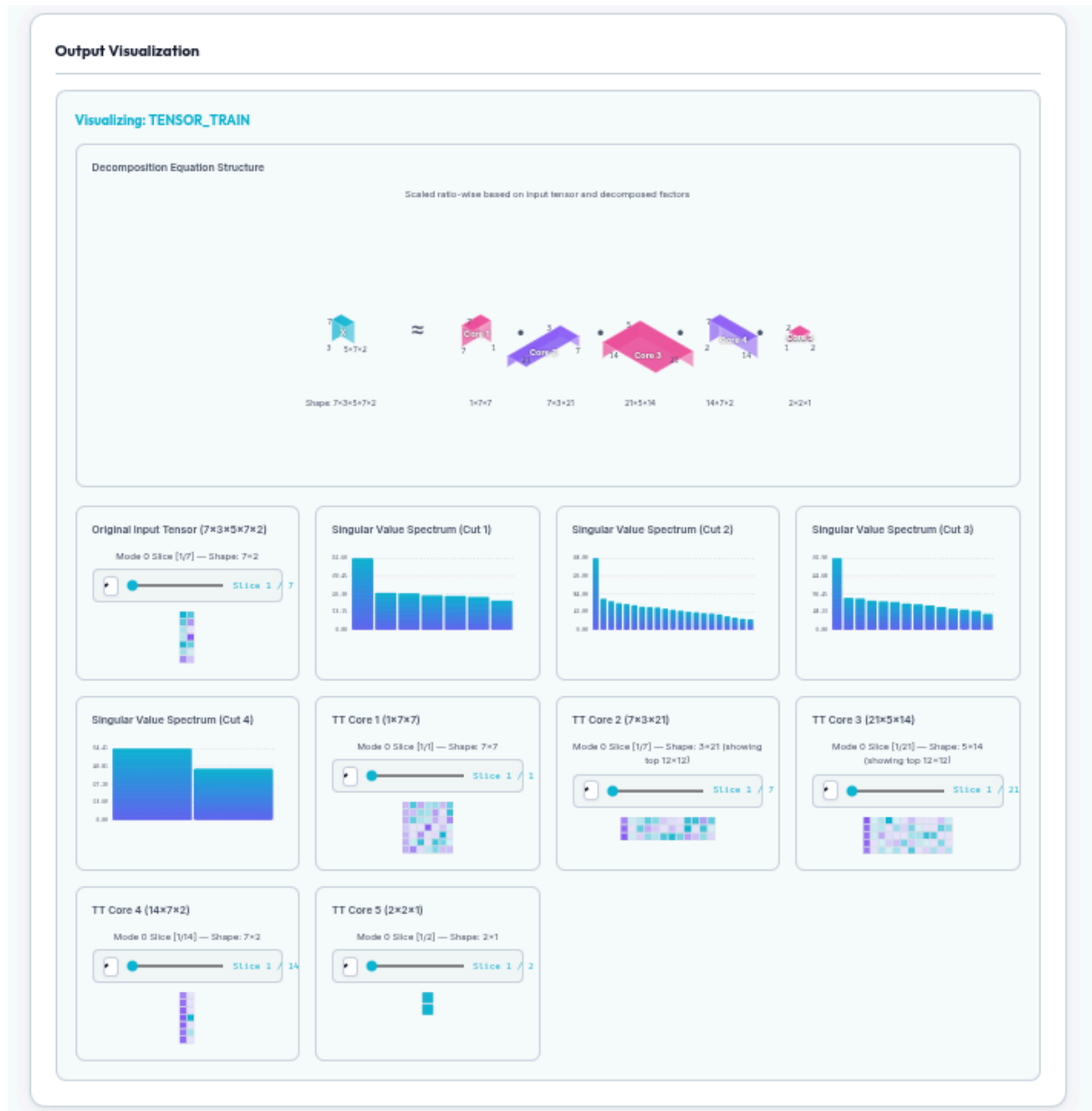


Figure - 13: Visualize (Tensor Train Decomposition)

7.2.4 Decompose Benchmark

Steps:

1. Enter a tensor via manual insertion, random input in the left panel or file input in the right panel.
2. Press the 'Benchmark & Compression' button in the right panel.
3. Benchmark analysis will be shown in the bottom section.

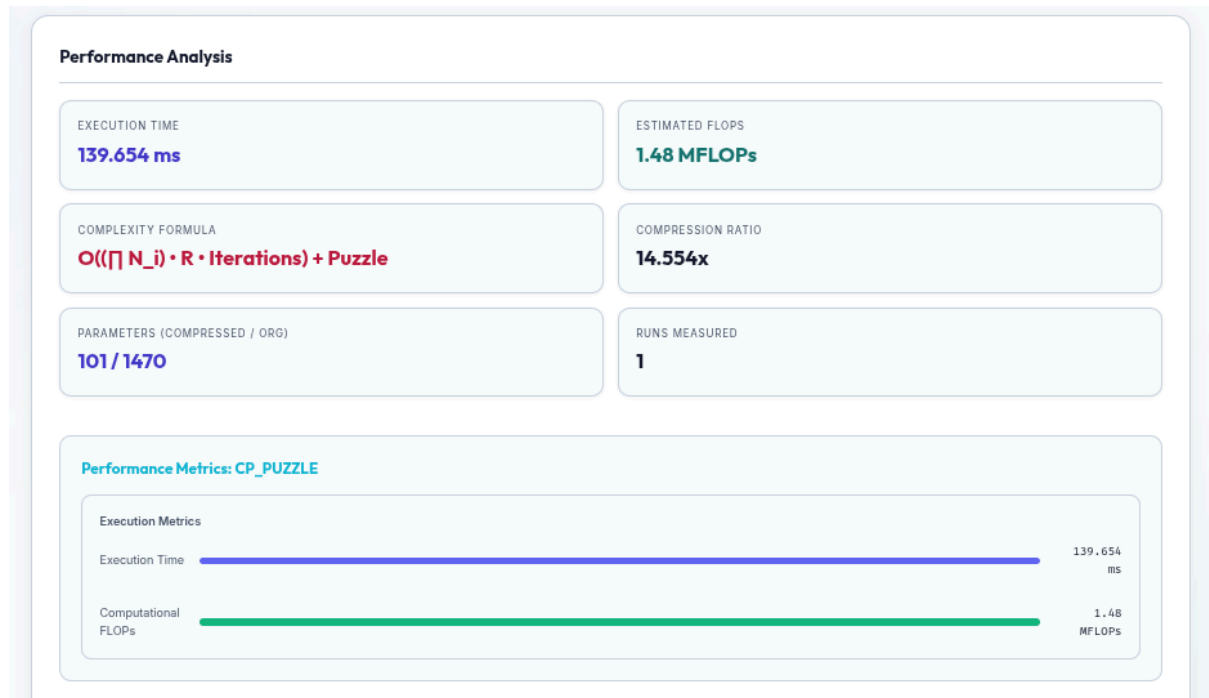


Figure - 14: Benchmark

7.2.5 Decompose Restruct

Steps:

1. Enter a tensor via manual insertion, random input in the left panel or file input in the right panel.
2. Press the 'Restruct & Accuracy' button in the right panel.
3. Accuracy analysis will be shown in the bottom section.

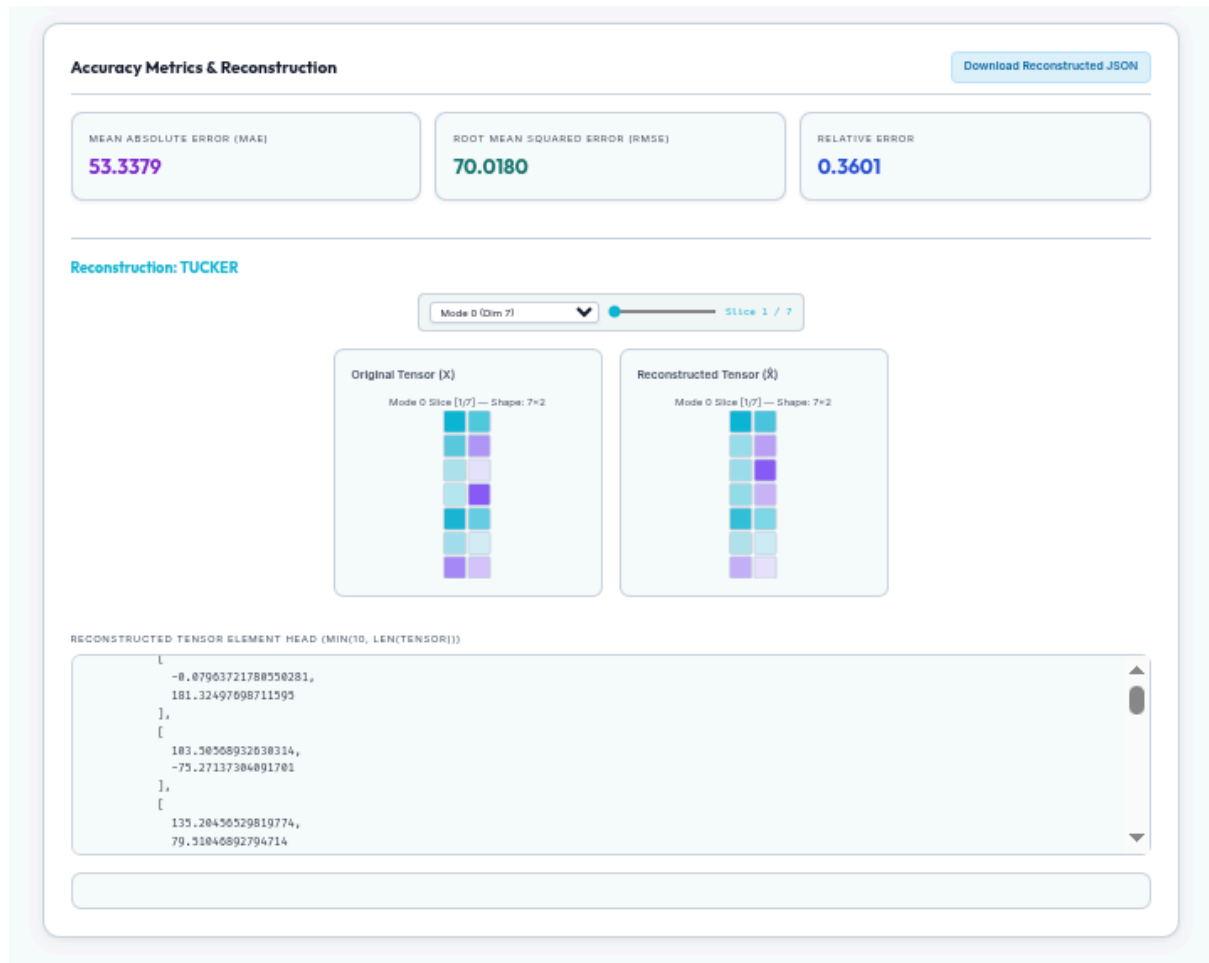


Figure - 15: Reconstruct

7.2.6 Compare All Methods

Steps:

1. Enter a tensor via manual insertion, random input in the left panel or file input in the right panel.
2. Press the 'Compare All Methods' button in the right panel.
3. Accuracy, execution time, compression values and graph will be shown in the bottom section.



Figure - 16: Comparison of Methods

7.3 Library Installation

Steps:

1. Install python in the operating system
2. Make a python environment via

```
> python -m venv .venv
```

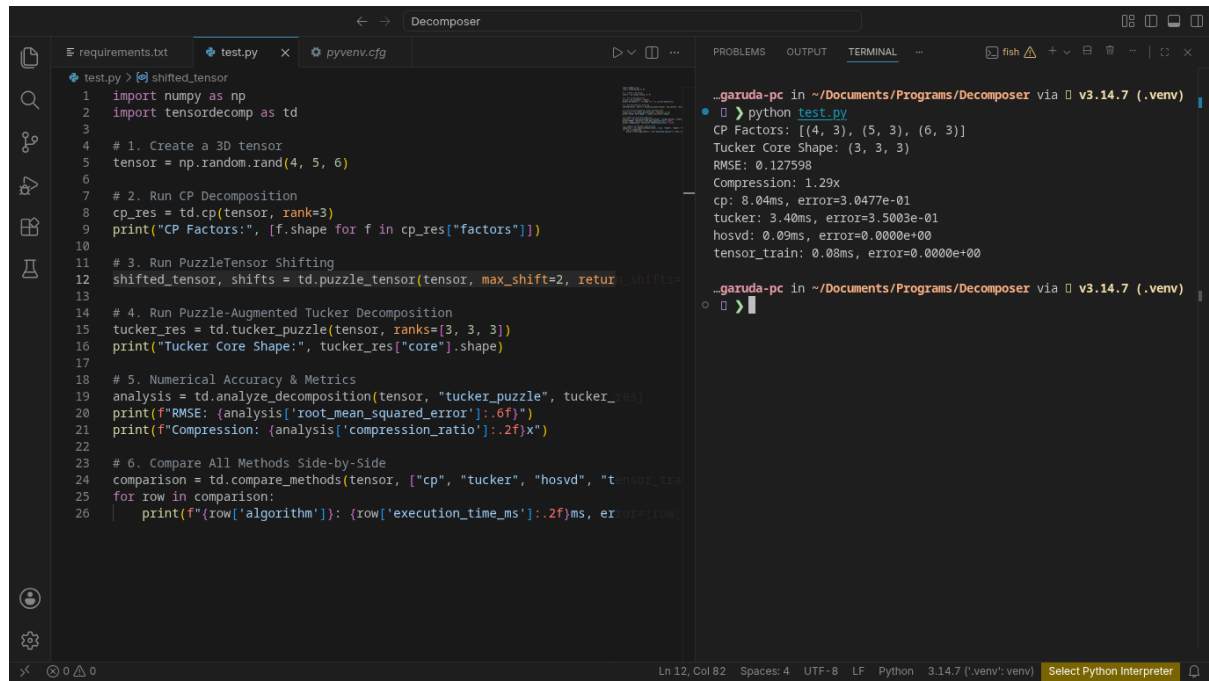
3. Install library command

```
> pip install tensordecomp
```

7.3.1 Library Functions

Function / Class	Description
<code>tensordecomp.cp(tensor, rank=R)</code>	CP-ALS decomposition. Returns {weights, factors, rank, shape}.
<code>tensordecomp.tucker(tensor, ranks=[R1,R2,R3])</code>	Tucker/HOOI decomposition. Returns {core, factors, ranks}.
<code>tensordecomp.hosvd(tensor, ranks=[R1,R2,R3])</code>	HOSVD decomposition. Returns {core, factors, ranks}.
<code>tensordecomp.tensor_train(tensor, ranks=[r1,r2])</code>	TT-SVD decomposition. Returns {cores, tt_ranks}.
<code>tensordecomp.puzzle_tensor(tensor, max_shift=2)</code>	Apply PuzzleTensor; returns (shifted_tensor, shifts).
<code>tensordecomp.cp_puzzle / tucker_puzzle / ...</code>	Puzzle-augmented variants. Same signature as base algorithm.
<code>tensordecomp.reconstruct_cp(weights, factors)</code>	Reconstruct full tensor from CP decomposition.
<code>tensordecomp.reconstruct_tucker(core, factors)</code>	Reconstruct full tensor from Tucker/HOSVD decomposition.
<code>tensordecomp.reconstruct_tt(cores)</code>	Reconstruct full tensor from Tensor Train cores.
<code>tensordecomp.analyze_decomposition(tensor, alg, res)</code>	Returns {rel_error, MAE, RMSE, compression_ratio, ...}.
<code>tensordecomp.benchmark_algorithm(tensor, alg)</code>	Returns {execution_time_ms, flops, flops_str, complexity}.
<code>tensordecomp.compare_methods(tensor, [alg1, alg2, ...])</code>	Runs all algorithms; returns comparative table list.
<code>tensordecomp.parse_tensor_input(text)</code>	Parses JSON/Python-style array string to <code>numpy.ndarray</code> .
<code>tensordecomp.run_algorithm(tensor, alg, **kwargs)</code>	Universal dispatch to any algorithm by name string.

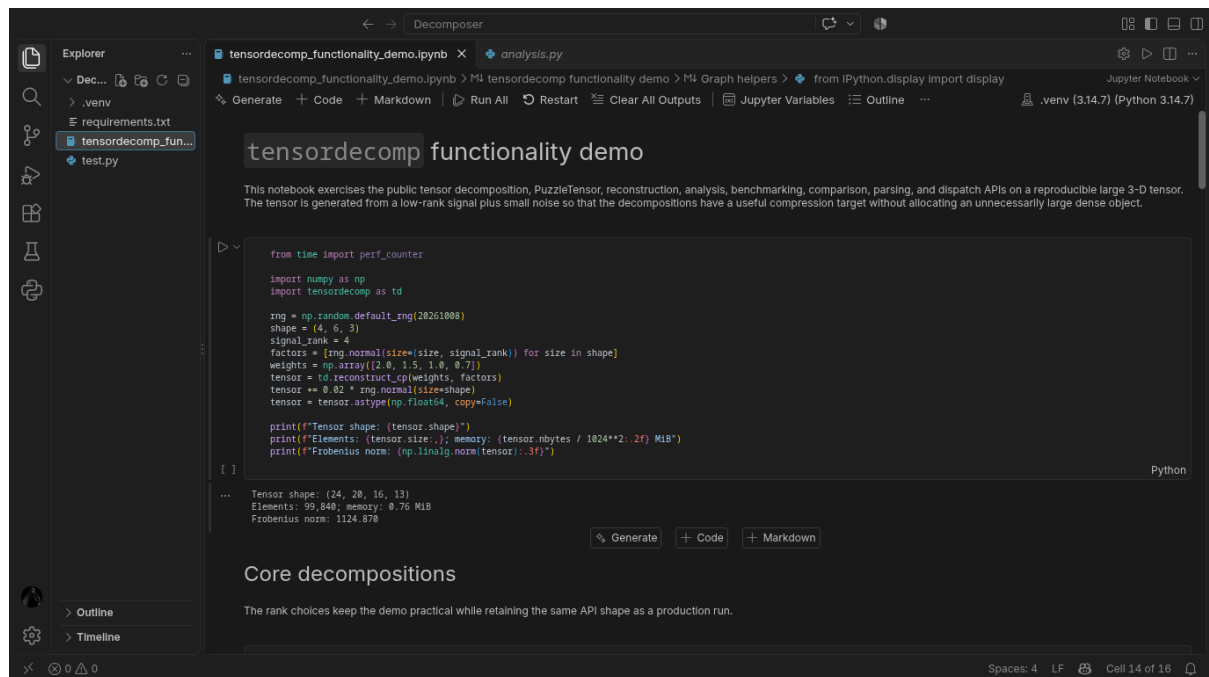
7.3.1 Library Implementations



```
1 import numpy as np
2 import tensordecomp as td
3
4 # 1. Create a 3D tensor
5 tensor = np.random.rand(4, 5, 6)
6
7 # 2. Run CP Decomposition
8 cp_res = td.cp(tensor, rank=3)
9 print("CP Factors:", [f.shape for f in cp_res["factors"]])
10
11 # 3. Run PuzzleTensor Shifting
12 shifted_tensor, shifts = td.puzzle_tensor(tensor, max_shift=2, return_shifts=True)
13
14 # 4. Run Puzzle-Augmented Tucker Decomposition
15 tucker_res = td.tucker_puzzle(tensor, ranks=[3, 3, 3])
16 print("Tucker Core Shape:", tucker_res["core"].shape)
17
18 # 5. Numerical Accuracy & Metrics
19 analysis = td.analyze_decomposition(tensor, "tucker_puzzle", tucker_res)
20 print(f"RMSE: {analysis['root_mean_squared_error']:.6f}")
21 print(f"Compression: {analysis['compression_ratio']:.2f}x")
22
23 # 6. Compare All Methods Side-by-Side
24 comparison = td.compare_methods(tensor, ["cp", "tucker", "hosvd", "tensor_train"])
25 for row in comparison:
26     print(f"row['algorithm']:\t {row['execution_time_ms']:.2f}ms, error={row['rmse']:.6f}")
```

```
garuda-pc in ~/Documents/Programs/Decomposer via v3.14.7 (.venv)
python test.py
CP Factors: [(4, 3), (5, 3), (6, 3)]
Tucker Core Shape: (3, 3, 3)
RMSE: 0.127598
Compression: 1.29x
cp: 8.04ms, error=3.0477e-01
tucker: 3.40ms, error=3.5003e-01
hosvd: 0.09ms, error=0.0000e+00
tensor_train: 0.08ms, error=0.0000e+00
```

Figure - 17: Decompose in .py file



```
from time import perf_counter
import numpy as np
import tensordecomp as td

rng = np.random.default_rng(20261008)
shape = (4, 6, 3)
signal_rank = 4
factors = [rng.normal(size=(size, signal_rank)) for size in shape]
weights = np.array([2.0, 1.5, 1.0, 0.7])
tensor = td.reconstruct_cp(weights, factors)
tensor += 0.02 * rng.normal(size=shape)
tensor = tensor.astype(np.float64, copy=False)

print(f"Tensor shape: {tensor.shape}")
print(f"Elements: {tensor.size}; memory: {tensor.nbytes / 1024**2:.2f} MiB")
print(f"Frobenius norm: {np.linalg.norm(tensor):.3f}")
```

```
Tensor shape: (24, 20, 16, 13)
Elements: 99,840; memory: 0.76 MiB
Frobenius norm: 1124.870
```

Figure - 18: Input in .ipynb file

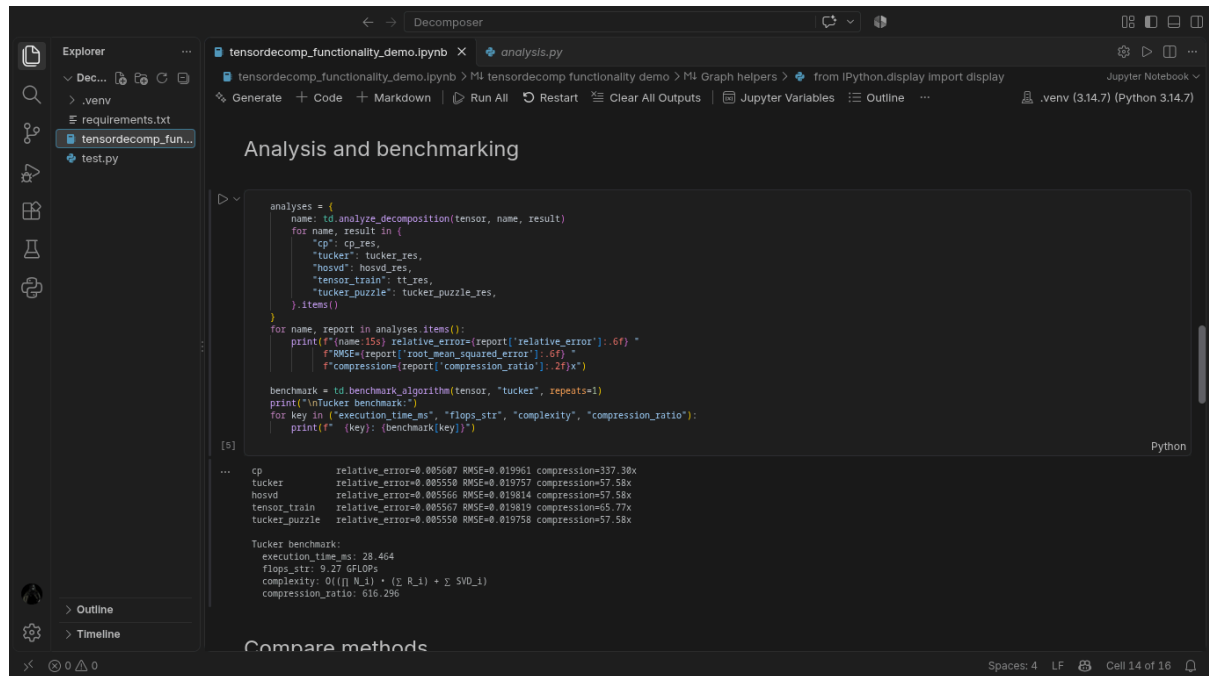


Figure - 19: Analysis in .ipynb file

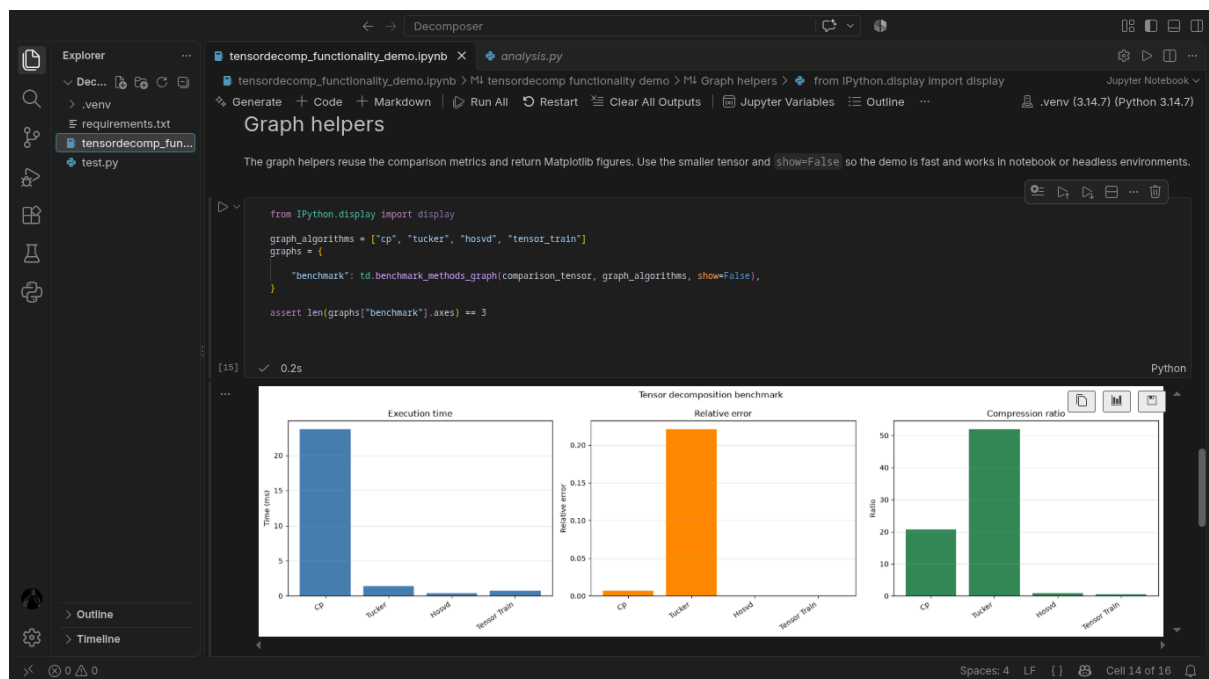


Figure - 20: Graph in .ipynb file

8. Public Accessible

8.1 Version Control Repository

Github url: <https://github.com/141Kamrul/Tensor-Composer>

8.2 Live url

The app is hosted in <https://td4.vercel.app/>

8.3 Library

Install command: `> pip install tensordecomp`

Dependencies of the library: `numpy, scipy libraries.`

9. Test Plan

9.1 Testing Strategy & Objectives

The testing strategy covers four levels:

- **Unit Tests:** Verify mathematical algorithms (CP, Tucker, HOSVD, Tensor Train, and PuzzleTensor augmentations) against numerical ground truth, ensuring reconstruction stability, correct factor dimensions, and error metrics.
- **Integration Tests:** Verify the full end-to-end data processing pipeline through Django views.
- **Library Function Tests:** Test library functionalities working properly in different setups and environments.
- **API / View Tests:** Verify HTTP GET and asynchronous AJAX POST requests, validating HTTP status codes.
- **End-to-End User Acceptance Tests (UAT):** Utilize Playwright (Chromium) to simulate real user interactions in both headed and headless browser environments, verifying dynamic UI controls, preset auto-insert chips, factor heatmap SVG rendering, dual-tensor side-by-side reconstruction navigation, synchronized slice sliders, benchmarking, and error panels.

9.2 Test Cases

9.2.1 Automated Unit, Integration & API Test Cases

TC-ID	Feature / Module	Input / Action	Expected Output	Status
TC-01	CP Decomposition	Execute CP on $3 \times 3 \times 3$ tensor with $R = 2$	Weights vector ($R = 2$) + 3 factor matrices (3×2)	PASS
TC-02	Tucker Decomposition (HOOI)	$3 \times 4 \times 5$ tensor with $ranks = [2, 2, 2]$	$2 \times 2 \times 2$ core tensor + 3 factor matrices	PASS
TC-03	Higher Order SVD (HOSVD)	$4 \times 4 \times 4$ tensor with default ranks	Orthogonal factor matrices + core tensor	PASS
TC-04	Tensor Train Decomposition	4D tensor with automated TT-ranks	Sequence of 3D TT-core tensors with matching bond dimensions	PASS
TC-05	Reconstruction Error Metrics	Decompose 3D tensor and run accuracy analysis	MAE, RMSE, Relative Error, and reconstructed preview generated	PASS
TC-06	FLOPs & Performance Benchmark	Run benchmark on Tucker decomposition	Execution time (ms), FLOPs count, formula string, and parameters	PASS
TC-07	Multi-Method Comparison	Trigger comparison across all 8 tensor algorithms	Comparative matrix populated with times, errors, and compression	PASS

TC-08	CP + PuzzleTensor	Execute <code>cp_puzzle</code> on 3D tensor	PuzzleTensor-augmented CP factors with spatial shift indices	PASS
TC-09	Alphabet / Non-numeric Input	POST <code>[["a", "b"], ["c", "d"]]</code>	HTTP 400 with float conversion error message	PASS
TC-10	Incomplete Bracket Syntax	POST <code>[[[1, 2], [3, 4]]]</code>	HTTP 400 with syntax format error message	PASS
TC-11	Corrupt Binary File Upload	Upload <code>.png</code> or binary <code>.bin</code> file	HTTP 400 with unsupported file format error message	PASS
TC-12	Valid File Upload	Upload valid UTF-8 <code>.txt</code> or <code>.json</code> tensor file	HTTP 200 with successful decomposition factors	PASS
TC-13	High-Order Tensor (5D)	$2 \times 2 \times 2 \times 2 \times 2$ 5D tensor input	Successful decomposition with 5 factor matrices	PASS
TC-14	Singleton Dimensions	$1 \times 5 \times 3$ tensor input	Decomposition executes stably without dimension crash	PASS
TC-15	Negative Rank Input	CP with $rank = -5$	Clamped to $\min_r rank = 1$; decomposition executes	PASS

TC-16	Extreme Rank Input	CP with $rank = 999$	Clamped to $max_r\ rank$; executes without memory leak	PASS
TC-17	HTTP GET Home View	GET request to /	HTTP 200 OK, <code>home.html</code> rendered with base context	PASS
TC-18	AJAX Decompose Request	POST valid tensor with <code>X-Requested-With: XMLHttpRequest</code>	HTTP 200 with JSON payload containing factors and analysis	PASS
TC-19	Input Mutual Exclusion	Populate textarea and select file simultaneously	Uploaded file takes precedence; manual textarea is cleared	PASS
TC-20	Rank Parsing Logic	Parse "5", "2, 3, 4", "[2, 3]", ""	Correctly parsed to <code>int</code> , <code>list[int]</code> , or <code>None</code>	PASS
TC-21	Theme Persistence	Toggle theme in navigation header	CSS custom properties switch; theme state stored	PASS
TC-22	Tucker + PuzzleTensor	Execute <code>tucker_puzzle</code> on $3 \times 3 \times 3$ tensor	Core tensor and factors computed with Puzzle partitioning	PASS
TC-23	HOSVD + PuzzleTensor	Execute <code>hosvd_puzzle</code> on $3 \times 3 \times 3$ tensor	Orthogonal factors and core computed via Puzzle mode cuts	PASS

TC-24	Tensor Train + PuzzleTensor	Execute <code>tensor_train_puzzle</code> on 4D tensor	TT cores computed with PuzzleTensor block acceleration	PASS
TC-25	Empty Tensor String	POST empty string "" as tensor input	HTTP 400 with "Tensor input is empty" error	PASS
TC-26	Ragged Tensor Array	POST <code>[[1, 2], [3, 4, 5]]</code> (unequal row lengths)	HTTP 400 with inhomogeneous sub-slice error	PASS
TC-27	1D Vector Input	POST <code>[1, 2, 3, 4]</code> (1D array)	HTTP 400 with minimum 2 dimensions requirement error	PASS
TC-28	Direct 2D Matrix Decompose	POST <code>[[1, 2], [3, 4]]</code> matrix	HTTP 200 with 2D factor decomposition matrices	PASS
TC-29	Reconstructed JSON Export	Analysis of reconstructed tensor	Reconstructed tensor JSON file created for user download	PASS
TC-30	Artifact File Download	GET request to <code>/download/<filename>/</code>	HTTP 200 with JSON attachment and Content-Disposition	PASS
TC-31	Path Traversal Protection	GET request to <code>/download/../../etc/passwd/</code>	Sanitized to basename; rejects unauthorized directory access	PASS

9.2.2 User Acceptance Test Cases

UAT-ID	User Journey / Workflow	User Action in Browser	Expected UI Behavior & Verification	Status
UAT-01	Page Load & Theme Switcher	Navigate to application URL; click theme toggle button	Header and hero title render; <code>data-theme</code> attribute toggles between light and dark modes	PASS
UAT-02	Preset Shape Auto-Insert Chips	Click chips: $3 \times 3 \times 3$, $4 \times 3 \times 2$, $4D$ (2^4), $4D$ ($3 \times 2 \times 3 \times 2$)	Textarea auto-populates with valid JSON; feedback badge displays "Inserted [shape]"	PASS
UAT-03	Dynamic Rank Hint Badges	Switch algorithm dropdown between CP, Tucker, and TT	Rank hint badge updates dynamically (e.g., "Rank R", " R_1 , R_2 , R_3 ", "TT-ranks")	PASS
UAT-04	CP Decomposition Journey	Auto-insert $3 \times 3 \times 3$ tensor, input rank 2, click Decompose	<code>#panel-decomposition</code> appears; factor matrices and JSON download button are visible	PASS
UAT-05	Tucker Decomposition Journey	Auto-insert $4 \times 3 \times 2$ tensor, input ranks 2, 2, 2, click Decompose	Decomposition output displays core tensor slices and 3 factor matrices	PASS

UAT-06	HOSVD Decomposition Journey	Auto-insert 3×3×3 tensor, select HOSVD, click Decompose	Output renders orthogonal factor matrices and core tensor slices	PASS
UAT-07	Tensor Train 4D Journey	Auto-insert 2×2×2×2 4D tensor, select TT, click Decompose	Output renders sequence of 4 TT-core tensors with internal bond ranks	PASS
UAT-08	PuzzleTensor Augmented Methods	Select and run cp_puzzle, tucker_puzzle, hosvd_puzzle, tensor_train_puzzle	All 4 augmented methods execute successfully; decomposition panels render	PASS
UAT-09	Interactive Visualization	Auto-insert tensor, select Tucker, click Visualize	#panel-visualization renders decomposition equation SVG and factor heatmaps	PASS
UAT-10	Dual-Tensor Reconstruction Viewer	Auto-insert 3×3×3 tensor, click Reconstruct & Accuracy	Side-by-side original (X) and reconstructed ($\hat{X}$) heatmaps render; slice slider updates badges	PASS
UAT-11	Synchronized Slice Slider Navigation	Drag slice range slider from slice 0 to slice 1	Both original and reconstructed cards synchronize views; badge shows "Slice 2 / 3"	PASS

UAT-12	Reconstructed Tensor Download	Run Restruct & Accuracy; verify download button	#btn-download-recon is active with valid download link for reconstructed JSON	PASS
UAT-13	Performance Benchmark Flow	Auto-insert tensor, select HOSVD, click Benchmark	#panel-benchmark renders execution time (ms), compression ratio, and chart canvas	PASS
UAT-14	Multi-Method Comparison Table	Click Compare All Methods on 3D tensor	#panel-comparison displays table with all 8 algorithms, times, errors, and ratios	PASS
UAT-15	Direct 2D Matrix Input	Enter <code>[[1, 2, 3], [4, 5, 6], [7, 8, 9]]</code> , click Decompose	2D matrix decomposed into factor matrices without dimensional errors	PASS
UAT-16	Valid File Upload Workflow	Select <code>.json</code> file via file picker, click Decompose	Textarea clears; uploaded tensor is processed and decomposed successfully	PASS
UAT-17	Error Handling: Empty Input	Clear textarea and click Decompose	Error panel appears with message: "Please enter a tensor manually or upload a tensor file"	PASS
UAT-18	Error Handling: Malformed JSON	Enter <code>[[[1, 2], [3, 4]</code> (missing closing bracket)	Error panel appears with message: "Incomplete	PASS

			tensor input: unclosed or unbalanced brackets"	
UAT-19	Error Handling: Ragged Arrays	Enter <code>[[1, 2], [3, 4, 5]]</code> (unequal row lengths)	Error panel appears with message: "Incomplete or ragged tensor input: rows have unequal lengths"	PASS
UAT-20	Error Handling: Non-Numeric Data	Enter <code>[["alpha", "beta"], ["gamma", "delta"]]</code>	Error panel appears with message: "could not convert string to float"	PASS
UAT-21	Error Handling: 1D Vector Input	Enter <code>[1, 2, 3, 4]</code> (1D vector)	Error panel appears with message: "requires a tensor with at least 2 dimensions"	PASS
UAT-22	Error Handling: Corrupt File Upload	Upload binary file (e.g., <code>.bin</code> containing non-UTF8 bytes)	Error panel appears with message: "Unsupported file format... Please upload a valid UTF-8 file"	PASS

9.2.3 Library Test Cases

Test ID	Module / Test Function	Library Action / Python Execution Call	Expected Output & Verification	Status

LTC-01	Package Exports test_library_exports	Check <code>hasattr(td, export)</code> for 15 top-level package functions (<code>cp</code> , <code>tucker</code> , <code>hosvd</code> , <code>tensor_train</code> , <code>puzzle_tensor</code> , <code>compare_methods</code> , <code>parse_tensor_input</code> , etc.)	All 15 required module attributes exist and are importable from <code>tensordecomp</code> .	PASS
LTC-02	Input String Parsing test_parse_tensor_input	<code>td.parse_tensor_input("[[[1, 2], [3, 4]], [[5, 6], [7, 8]]]")</code>	Returns NumPy ndarray of shape (2, 2, 2) with value <code>arr[0, 1, 0] == 3.0</code> .	PASS
LTC-03	CP Decomposition test_cp_decomposition	<code>td.cp(tensor, rank=2)</code> on array of shape (2, 3, 4)	Returns <code>method="cp"</code> and 3 factor matrices of shapes (2, 2), (3, 2), and (4, 2).	PASS
LTC-04	Tucker Decomposition test_tucker_decomposition	<code>td.tucker(tensor, ranks=[2, 2, 2])</code> on shape (2, 3, 4)	Returns <code>method="tucker"</code> , core tensor of shape (2, 2, 2), and 3 factor matrices.	PASS
LTC-05	HOSVD & Reconstruction test_hosvd_and_reconstruction	<code>td.hosvd(tensor)</code> followed by <code>td.reconstruct_tensor("hosvd", result)</code>	Reconstructed tensor matches original (2, 3, 4) array within <code>rtol=1e-4</code> , <code>atol=1e-4</code> .	PASS

LTC-06	Tensor Train (TT-SVD) test_tensor_train	td.tensor_train(tensor, max_rank=3) followed by reconstruct_tensor() 	Returns 3 TT core tensors; reconstructed array shape matches original shape (2, 3, 4).	PASS
LTC-07	Puzzle Shifting Roundtrip test_puzzle_tensor_roundtrip	td.puzzle_tensor(tensor) followed by td.invert_puzzle_tensor(shifted, shifts) 	Restored array matches original tensor exactly within tolerance (rtol=1e-5, atol=1e-5).	PASS
LTC-08	Puzzle-Augmented Methods test_puzzle_augmented_methods	Execute td.run_algorithm() for cp_puzzle, tucker_puzzle, hosvd_puzzle, tensor_train_puzzle 	Returns is_puzzle=True, slice shifts, valid compression_ratio > 0, and root_mean_squared_error.	PASS
LTC-09	Comparative Analysis test_compare_methods	td.compare_methods(tensor, ["cp", "tucker", "hosvd", "tensor_train"]) 	Returns 4 summary dictionaries containing compression_ratio, relative_error, and execution_time_ms.	PASS
LTC-10	Multilinear Rank Validation test_mode_and_multilinear_ranks	Test scalar rank for CP, mode ranks [2, 2, 3] for Tucker, HOSVD, and TT ranks [1, 2, 2, 1] 	Correctly computes and validates min_ranks, max_ranks, and factor tensor dimensions	PASS

			across all 4 algorithms.	
LTC-11	Complexity Formula Formatting test_complexity _formula_produc t_of_n	td.get_complexity_ formula(shape, method) for CP, Tucker, and Tensor Train	Returns formatted mathematical strings utilizing product notation ($\prod N_i$) instead of $N_1 N_2 N_3$.	PASS

10. Conclusion

Tensor Decomposer successfully delivers a unified, interactive, publicly deployed platform for higher-dimensional tensor analysis. The system implements eight tensor decomposition algorithms (including four PuzzleTensor-augmented variants) all accessible through a modern web interface without local installation.

The primary objective of the project is helping students better understand decomposition of tensors and letting researchers choose the right method for their problem by comparing accuracy, execution time and compression. The web app and library can hope to overcome those unique situations.

Furthermore, this report will help users better understand the project, its objectives, reasoning, requirements and outcomes.

11. References

- [1] Kolda, T. G., & Bader, B. W. (2009). Tensor decompositions and applications. *SIAM Review*, 51(3), 455–500. <https://doi.org/10.1137/07070111X>
- [2] Oseledets, I. V. (2011). Tensor-train decomposition. *SIAM Journal on Scientific Computing*, 33(5), 2295–2317. <https://doi.org/10.1137/090752286>
- [3] Park, J., et al. (2025). PuzzleTensor: A Method-Agnostic Data Transformation for Compact Tensor Factorization. *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD 2025)*.
<https://doi.org/10.1145/3690624.3709227>
- [4] Github link: <https://github.com/141Kamrul/Tensor-Composer>
- [5] Live url: <https://td4.vercel.app/>